GPU reductions

[M2 DiPaC] GPU Programming, Université Paris-Saclay – Session 6

Oguz Kaya

Assistant Professor
Université Paris-Saclay and LISN, Gif-sur-Yvette, France

université PARIS-SACLAY

Objectives

- Know what a **reduction** is, and why it is an awkward computation for a GPU: the parallelism shrinks at every step, and there is one operation per element loaded
- Write the **block reduction**, in global memory first, then in shared memory, and improve it step by step with what we have: coalescing (session 3), the mapping of the threads on the warps (session 4), several elements per thread, the last warp alone
- **Finish** a reduction: the partials, a second launch, **atomics**
- Measure a reduction against the right yardstick: the **bandwidth** of the memory

Outline

1. What a reduction is
2. The block reduction, improved step by step
3. Summary

Outline

1. What a reduction is
2. The block reduction, improved step by step
3. Summary

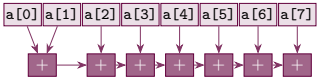
Recap: the three tools of this session

- **Coalescing** (session 3): a warp's load is one request for 32 addresses, served in 32-byte sectors; 32 consecutive floats cost 4 sectors, every byte used
- **Divergence** (session 4): the 32 threads of a warp execute together; a warp executes an instruction as soon as *one* of its threads needs it, the others idle. A branch costs nothing when the whole warp takes the same side
- **Shared memory** and `__syncthreads()` (session 5): the threads of a block share an on-chip array; a thread that reads what *another* thread wrote waits at a barrier first, and every thread of the block must reach every barrier

Today's problem uses all three at once: the **sum** of n floats, $s = \texttt{a[0]} + \texttt{a[1]} + \cdots + \texttt{a[n-1]}$. The same shape serves the minimum and the maximum: a **reduction**

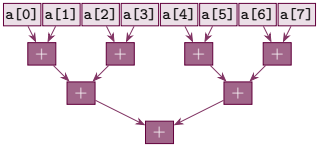
The sum of n numbers: a chain, or a tree

sequentially: a chain of $n-1$ additions



each addition waits for the previous one:
no parallelism at all

as a tree: $\log_2 n$ levels



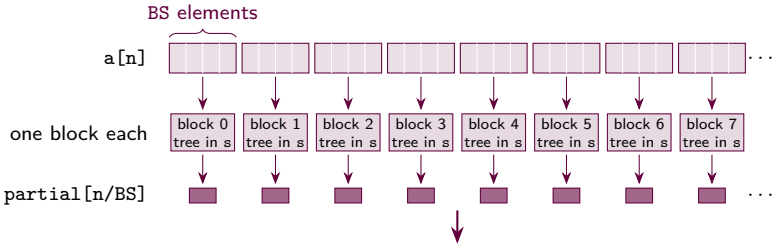
level 1: 4 additions, in parallel

level 2: 2 additions

level 3: 1 addition

- Sequentially: $n-1$ additions, each one waiting for the previous one. As a tree: $\log_2 n$ **levels** of independent additions – 24 levels for 16 million elements
- Two difficulties for a GPU: the number of additions **halves at each level** (thousands of threads at level 1, one at the last); and the whole computation is **one addition per element loaded**: no reuse, nothing to compute while waiting
- So the yardstick is the **bandwidth**: a reduction that reads n floats once, at the speed of the memory, is as fast as it can be. On the T4: ≈ 250 GB/s, 16 M floats in 0.27 ms

On a GPU: one block per chunk, then the partials



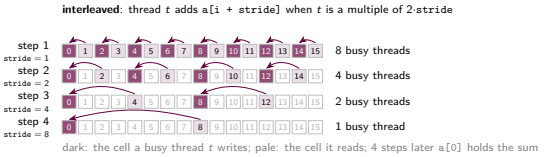
the partials, reduced afterwards: by the CPU,
by a second launch of the same kernel, or by one atomic per block – the result

- Blocks cannot wait for each other inside a kernel (session 5): the tree is cut in two. Each block reduces its BS elements to **one partial sum**, through a tree with a barrier at each step; the n/BS partials are reduced afterwards
- The partials: the **CPU** sums them (fine when there are few), or the same kernel is **launched again** on them, or each block adds its partial to the result with an **atomic** – slide 11
- The rest of the session: five block-reduction kernels, each one fixing a flaw of the one before

Outline

1. What a reduction is
2. The block reduction, improved step by step
3. Summary

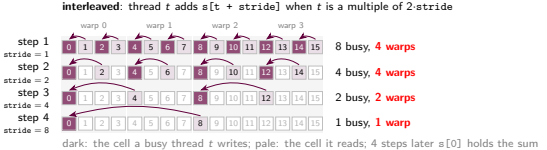
Kernel 1: the tree in global memory




```
1  __global__ void reduceGlobal(float *a,
2      float *partial, int n)
3  {
4    int t = threadIdx.x;
5    int i = blockIdx.x * BS + t;
6    for (int stride = 1; stride < BS; stride *= 2) {
7      if (t % (2 * stride) == 0) {
8        a[i] += a[i + stride];
9      }
10     __syncthreads();
11   }
12   if (t == 0) partial[blockIdx.x] = a[i];
13 }
```

- The tree of slide 4 on the array itself: thread `t` of block `b` owns `a[i]`, $i = b \cdot \mathrm{BS} + t$. At step `stride` $= 1, 2, 4, \ldots$ it adds `a[i + stride]` to `a[i]` if `t` is a multiple of 2·`stride`; a barrier per step; thread 0 writes its `a[i]` to `partial[b]`. In place: `a` is destroyed
- **Uncoalesced**: the busy threads of a warp are 2, 4, 8, ... words apart, so a warp's load fetches 32 floats to use 16, then 8, 4, ...; and every step is a round trip to global memory: read, add, write back
- On the T4: 1.2 ms, **56 GB/s** – a quarter of the memory's bandwidth

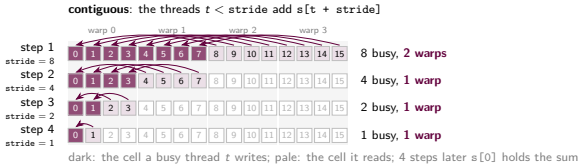
Kernel 2: the same tree in shared memory




```
1  __shared__ float s[BS];
2  int t = threadIdx.x;
3  int i = blockIdx.x * BS + t;
4  s[t] = (i < n) ? a[i] : 0.0f;   // 0: identity
5  __syncthreads();
6  for (int stride = 1; stride < BS; stride *= 2) {
7    if (t % (2 * stride) == 0) {
8      s[t] += s[t + stride];
9    }
10   __syncthreads();
11 }
12 if (t == 0) partial[blockIdx.x] = s[0];
13 }
```

- **The fix**: a coalesced load into `s[t]` (a warp reads 32 consecutive floats; 0 past the end), one barrier, then the same tree in shared memory
- No coalescing problem any more. **Still a problem?** Look at the warps
- A warp executes a step as soon as *one* of its threads is busy, and the busy threads sit in **every warp** until `stride` reaches 32: at step 5, 8 warps run for 8 additions – one lane in 32 at work
- $8 \times 5 + 4 + 2 + 1 = 47$ warp-steps for 255 additions: the SM is busy with **nearly empty warps**

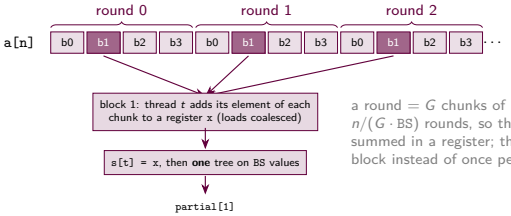
Kernel 3: the tree with contiguous threads




```
1  __shared__ float s[BS];
2  int t = threadIdx.x;
3  int i = blockIdx.x * BS + t;
4  s[t] = (i < n) ? a[i] : 0.0f;
5  __syncthreads();
6  for (int stride = BS / 2; stride > 0; stride /= 2) {
7    if (t < stride) {
8      s[t] += s[t + stride];
9    }
10   __syncthreads();
11 }
12 if (t == 0) partial[blockIdx.x] = s[0];
13 }
```

- Kernel 3 packs the busy threads at the **front**: `stride` $= \mathtt{BS}/2, \ldots, 1$, the threads `t < stride` add `s[t + stride]`. Whole warps **retire**: $4 + 2 + 6 \times 1 = 12$ warp-steps instead of 47, the last 6 inside warp 0 alone, and no %
- Same load, same 8 steps and 8 barriers: what changes is *which* threads add
- The words a warp reads, `s[t + stride]`, are consecutive: **1-way** in the banks (kernel 2's too: every other word, 16 banks)

Kernel 4: several elements per thread, then the tree



a round = G chunks of BS elements, one per block; $n/(G \cdot \mathrm{BS})$ rounds, so that many elements per thread, summed in a register; the tree and its barriers run once per block instead of once per BS elements; G partials remain

```
1  __global__ void reduceMulti(const float *a,
2      float *partial, int n)
3  {
4    __shared__ float s[BS];
5    int t = threadIdx.x;
6    float x = 0.0f;
7    for (int i = blockIdx.x * BS + t; i < n;
8          i += gridDim.x * BS) {
9      x += a[i];                    // coalesced
10   }
11   s[t] = x;
```

- Kernels 2 and 3 run a tree of 8 steps and 8 barriers for **every 256 elements**, and load nothing meanwhile
- Fewer blocks, **more elements per thread**: $G = 1024$ blocks for 16 M elements, each thread sums 64 elements in a **register** (a *grid-stride* loop, loads coalesced), then *one* tree per block, kernel 3's: amortised 64 times over, and only G partials remain
- The listing stops before the tree

Kernel 5: the tree unrolled, the last warp alone


```
1   __syncthreads();
2 #if BS >= 1024
3   if (t < 512) { s[t] += s[t + 512]; } __syncthreads();
4 #endif
5 #if BS >= 512
6   if (t < 256) { s[t] += s[t + 256]; } __syncthreads();
7 #endif
8 #if BS >= 256
9   if (t < 128) { s[t] += s[t + 128]; } __syncthreads();
10 #endif
11 #if BS >= 128
12   if (t <  64) { s[t] += s[t +  64]; } __syncthreads();
13 #endif
14   if (t < 32) {                    // warp 0 only
15     s[t] += s[t + 32]; __syncwarp();
16     if (t < 16) { s[t] += s[t + 16]; } __syncwarp();
17     if (t <  8) { s[t] += s[t +  8]; } __syncwarp();
18     if (t <  4) { s[t] += s[t +  4]; } __syncwarp();
19     if (t <  2) { s[t] += s[t +  2]; } __syncwarp();
20     if (t == 0) partial[blockIdx.x] = s[0] + s[1];
21   }
22 }
```

step	busy threads	warps	barrier
1	128	4	`__syncthreads()`
2	64	2	`__syncthreads()`
3–8	32, 16, 8, 4, 2, 1	1	`__syncwarp()`

- **Unrolled**: kernel 4's loop written out, one line per step; `#if BS >= ...` keeps the steps of this block size (`BS` is a compile-time constant): no counter, no test
- From step 3 on only **warp 0** is busy: in kernel 4 the 7 others wait at 6 block barriers for nothing. **Warp-synchronous**: the threads `t` < 32 do the last 6 steps alone, without `__syncthreads()`; `__syncwarp()` keeps the warp in step (required since Volta: the threads of a warp may run ahead of each other)
- The listing: lines 2–13, the tree down to `stride` = 64; lines 14–21, warp 0 alone

The partials: the CPU, a second launch, or an atomic

- **On the CPU**: copy the G partials back, add them up. Fine for $G = 1024$ (0.01 ms); a third of the kernel's time for 65 536
- **A second launch**: the same kernel, one block, on the array of partials (it works on any n, so on G values). +0.007 ms
- **An atomic**: thread 0 of each block adds its partial to `*result`: no partials, no second launch

```
if (t == 0) atomicAdd(result, s[0] + s[1]);
```

- `atomicAdd(p, v)`: `*p += v` as **one indivisible** read-modify-write by the memory system: two threads adding to the same word cannot lose each other's update. Also `atomicMin`, `atomicMax`, ... – on `int`, not on `float`
- The price: the updates of **one address are serialised**. One per block: G of them, invisible. One per *element*:

```
if (i < n) atomicAdd(result, a[i]);
```

16 M additions on one word, one after the other: **33 ms** instead of 0.27. An atomic finishes a reduction, it does not replace one

Measured on the T4: the bandwidth

kernel	blocks	partials	ms	GB/s
1 global memory, in place	65 536	65 536	**1.19**	**56**
2 interleaved, shared memory	65 536	65 536	0.57	118
3 contiguous	65 536	65 536	0.56	119
4 several elements per thread	1 024	1 024	0.27	**250**
5 unrolled, last warp alone	1 024	1 024	0.27	**251**
5 + second launch	1 024 + 1	0	0.28	244
6 one `atomicAdd` per block	1 024	0	0.27	250
one `atomicAdd` per element	65 536	0	**33.2**	**2**
read a once, compute nothing	1 024	–	0.27	250

`src/reduce.cu`, $n = 2^{24}$ floats (64 MB), BS $= 256$; Tesla T4 (cuda.godbolt.org); the kernel alone, the partials not counted. GB/s: the bytes of a divided by the time. The last line is the ceiling.

- Kernel 1 rewrites the array at every step: 56 GB/s. Kernel 2 (one coalesced read, the tree on chip) doubles it, yet stops at **half** the bandwidth, and 3 gains *nothing* over 2: a block loads 1 KB, then runs 8 steps without loading, and a step costs its latency (load, add, store, barrier) whether 8 warps or 1 add
- Kernel 4 reaches the **ceiling**: the loads never stop, the tree is 1/64 of the work. Kernel 5 halves the tree's own cost (measured apart: 1.2 $\rightarrow$ 0.6 μs per block), already invisible here
- The lesson: **keep the memory busy**; the tree is a detail once it is amortised

Two more things to know

- **Floating point**: the tree adds the numbers in another order than the sequential loop, and `float` addition is not associative: the two results differ in the last bits (a tree is in fact the *more* accurate of the two). Check a sum with a **tolerance**, $|s - s_{\text{ref}}| \leq 10^{-5}|s_{\text{ref}}|$, never with ==. A minimum, a maximum, an integer sum are exact whatever the order
- **Other reductions**: the same kernels with another operation and another identity – `fminf` and `FLT_MAX` for the minimum, `fmaxf` and `-FLT_MAX` for the maximum. Only the sum has a float atomic (`atomicMin`, `atomicMax` take integers): the lab's minimum finishes with a second launch

Why `BS` a power of two? The tree halves the busy threads at every step: $96 \to 48 \to 24 \to 12 \to 6 \to 3$, and 3 cannot be halved – an element would be left over. A power of two keeps every step exact; otherwise, pad `s` with the identity up to the next power of two.

Outline

1. What a reduction is
2. The block reduction, improved step by step
3. Summary

What to remember

- A **reduction** (sum, min, max) reads every element once and does one operation per element: **bandwidth-bound**. Its yardstick is the GB/s of a plain read of the array
- **The block reduction**: a coalesced load into `s`, barrier, then a tree of $\log_2$ `BS` steps with a barrier after each – not on the array itself (uncoalesced, rewritten at every step); `BS` a power of two; the identity of the operation past the end of the array
- **The mapping**: busy threads packed at the front (`t < stride`, `s[t] += s[t + stride]`), so that whole warps retire and the last steps stay inside one warp – then `__syncwarp()` replaces `__syncthreads()` for those steps
- **What really matters**: several elements per thread (a grid-stride loop, coalesced loads, additions in a register) before *one* tree per block: the memory never idles, the tree is amortised, few partials remain
- **The partials**: the CPU, a second launch of the same kernel, or one atomic per block. An atomic is a serialised read-modify-write: one per block is free, one per element is a disaster
- **Float sums** depend on the order: compare with a tolerance

This week's lab: tp5

One program, `find-minimum.cu`: the **minimum** of an array of n floats, kernels 2 to 5 of the lecture with `fminf` and `FLT_MAX`. The host code is given (the array, a -1 hidden in it, the timing, the check, the bandwidth); a read-only kernel gives the ceiling. Five versions to write, in order:

1. **interleaved** tree in shared memory, one element per thread, the CPU finishes the partials
2. **contiguous** tree
3. **several elements per thread** (grid-stride loop) before the tree, G blocks
4. the **last warp** alone, with `__syncwarp()`
5. the partials finished by a **second launch** of version 4: the CPU reads one value

Compare the GB/s of the five with the read-only kernel's, on godbolt's T4 or a lab GPU. Then the solutions of tp3 and tp4, and time to finish them.

Contact

Oguz Kaya
Université Paris-Saclay and LISN, Gif-sur-Yvette, France
oguz.kaya@universite-paris-saclay.fr
www.oguzkaya.com

Course material: `github.com/kayaogz/2627-m2dipac-gpuprogramming`